Separating repeated-item success from transfer in adaptive programming practice

An auditable learner model and reinforcement-learning study

Abstract

Adaptive programming recommenders need to distinguish correct execution, response prediction and the value of selecting an exercise. We develop a reproducible prototype combining twelve executable Python tasks, a Bayesian learner-state filter and a Double Deep Q-Network policy. A shared catalog and numerical parity checks connect the trained policy to the application. The controlled evaluation includes seven policies, five synthetic environments, three training seeds and 31,500 rollouts involving 1,500 distinct simulated learners. Accounting for repeated-item familiarity reduces validation negative log-likelihood in all three fits. However, this predictive improvement does not consistently improve exercise selection. In nominal dynamics, the repeat-aware policy achieves 17.91 percentage points of expected transfer-probe gain, compared with 18.85 for the naive variant and 19.16 for a weakest-skill rule. The paired difference against weakest skill is -1.24 points, with a conditional 95% interval of [-2.27, -0.21]. A repair-only negative control produces zero transfer gain despite substantial practice correctness. The study contributes an inspectable implementation and a bounded negative result: predictive gains alone do not establish recommendation value. The estimates are conditional on declared simulator assumptions and fixed trained models; they do not measure human learning effects or establish state-of-the-art performance.

Keywords: adaptive learning; programming education; knowledge tracing; reinforcement learning; simulation; reproducibility

1. Introduction

Repeated submissions are a defining feature of programming practice. A learner can repair one solution until it passes a test suite, yet remain unable to solve an unfamiliar task that requires the same underlying idea. An exercise recommender therefore faces a measurement problem: successful execution, predictable responses and transferable learning are related quantities with different meanings. Optimizing one does not automatically optimize the others.

Knowledge tracing supplies a history-dependent representation for personalization, but its usual predictive target is a subsequent observed response. The educational decision is different: which eligible exercise should the learner attempt next? The distinction is consequential when repetition improves item-specific familiarity, when knowledge states are imperfectly estimated, or when the reward favors immediate correctness. A complete implementation must also preserve the identity of the exercise on which a model was trained when that model is used in an application.

We address these issues with an executable research prototype and a controlled simulator benchmark. The application grades authored Python tasks, updates a learner-state estimate and selects a subsequent task through a learned policy. The benchmark separates practice responses from independent, unpractised probes. Historical programming-log forecasting is retained as separate evidence; those logs do not provide matched trajectories for the authored task bank and are not used to claim that the deployed recommender improves human learning.

The investigation asks three questions. RQ1: Does explicitly representing repeated-item familiarity improve next-response prediction? RQ2: Does this improvement produce higher transfer-oriented

policy value under a fixed training budget? RQ3: How sensitive are the conclusions to the reward and to changes in the simulated learning process?

The contributions are: an auditable connection between executable tasks and learned selection; a controlled comparison that retains unsuccessful as well as favorable contrasts; and a reproducible reporting path from frozen rollout records to application traces, tables and figures. Neither the Bayesian filter nor Double DQN is proposed as a new algorithm. The contribution is an implementation and evaluation study with a clearly delimited claim, rather than a claim of superior tutoring effectiveness.

2. Related work

Knowledge tracing originated as a model of procedural knowledge acquisition (Corbett & Anderson, 1994). Deep Knowledge Tracing instead represents response histories through recurrent neural networks (Piech et al., 2015). Minn (2022) discusses how knowledge assessment depends on the student model, domain representation and available observations. These distinctions motivate our separation of observed correctness, model beliefs and policy outcomes. They do not establish that our three author-assigned skill groups measure human cognitive constructs.

Programming-specific models can exploit information inside the submitted program. Code-DKT augments correctness histories with code-derived features (Shi et al., 2022). Our application executes code to provide feedback, but the learner-state model consumes only the item identifier and binary whole-task outcome. The program, number of passed cases, hint use and time spent writing are not model features. This choice makes the current information boundary explicit and leaves code-aware diagnosis as an extension.

Exercise recommendation already combines knowledge tracing and reinforcement learning. ExRec integrates question semantics, knowledge tracing and policy optimization (Ozyurt et al., 2025). UniER provides a common benchmark for item-level and path-level recommendation (Cheng et al., 2026). These works rule out novelty claims based simply on combining a student model with an RL algorithm. Our comparison is restricted to the seven implemented policies under a common local protocol; ExRec and UniER were reviewed but not reproduced, and their reported scores are not directly comparable to ours.

Girard et al. (2026) investigate adaptive instructional policies using logged interactions and a contextual-bandit formulation. Such work is relevant when prospective intervention is unavailable, but an offline policy-value estimate requires justified modeling and evaluation assumptions. Matched histories and assignment propensities for our twelve tasks are unavailable. We consequently use explicit simulation for policy evaluation and retain observational forecasting claims separately. Double DQN is used as an established value-based learning method (van Hasselt et al., 2016).

3. System and problem formulation

3.1. Task bank and information boundary

The bank contains 12 Python tasks and 96 executable reference cases. Each task has a statement, examples, starter function, hints, test cases, an author-assigned primary skill and a difficulty level. The skill groups are selection and accumulation, memory and data structures, and state updates. The shared catalog supplies the same task identifiers and action order to training and browser inference. A SHA-256 mismatch disables learned selection. This establishes software compatibility, not transfer from a simulator to a human population.

Figure 1 summarizes training and application operation. At decision time, the recommender uses only observations already available in the chronological history. Passed and failed grading outcomes are included; identical source code for the same task contributes at most one model observation. Execution errors, cancellation and malformed observations are excluded. The displayed number of passed tests remains feedback from the grader, whereas the model target is binary success on the entire task.

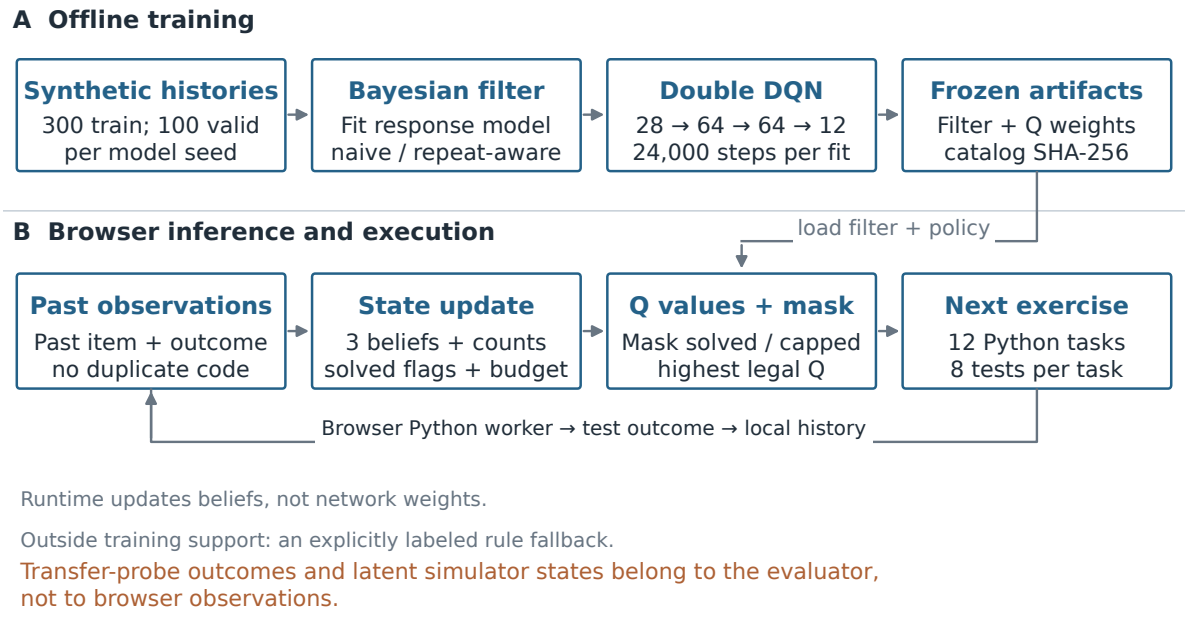


Figure 1. Training artifacts and browser decision flow. Only retained past responses enter the learner-state update. The application does not observe latent simulator skills.

The system does not infer psychological mastery from a test count. Its three belief values are internal model variables with a synthetic prior of 0.3. A new user receives a prior-based recommendation, which is explicitly labeled as lacking individual evidence. New observations update the state; browser interactions do not retrain the policy weights.

3.2. Belief-state representation

Let b(k) denote the current belief for skill k, n(a) the previous count for task a, z(a) its solved flag and t the number of retained observations. The policy input contains 28 values: three beliefs, twelve capped counts divided by three, twelve solved flags and the remaining episode budget divided by twelve. A finite belief representation approximates a partially observed decision process; it is not proof that human learning is Markovian.

For every observation, the filter first applies forgetting f = 0.005 to all beliefs. The active skill then receives a learning transition with rate l(1 + 0.15d), where d is the authored difficulty. Item familiarity is rho = m(1 - exp(-n(a))). The response emissions for a learned and unlearned skill are shown in Equation 1; g, s and m are fitted guess, slip and familiarity parameters.

$$q_1 = 1 - s - 0.05d + (s + 0.05d)\rho,$$
$$q_0 = g - 0.05d + (1 - g + 0.05d)\rho. \tag{1}$$

If b* is the active belief after its transition, the predicted response probability is p = b*q1 + (1 - b*)q0. After observing y, the active belief receives the Bayesian update in Equation 2; other beliefs retain their forgetting update. Prediction precedes use of the current response label.

$$b^{+} = \frac{b^{*} q_1^{y} (1 - q_1)^{1 - y}}{p^{y} (1 - p)^{1 - y}}. \tag{2}$$

The naive filter fixes familiarity at zero. The repeat-aware filter searches m in {0, 0.4, 0.7}; both search learning in {0.04, 0.10, 0.16}, guess in {0.16, 0.24} and slip in {0.08, 0.16}. Each fit minimizes training response negative log-likelihood over this fixed grid. Validation is descriptive and is not used to select a checkpoint after examining policy outcomes.

3.3. Action space, reward and Double DQN

An action selects a specific task. The legal action set excludes solved tasks and tasks with three prior retained observations. The greedy policy selects the highest Q value among legal actions, breaking ties by action order. The 28-64-64-12 network has 6,796 parameters and rectified linear hidden layers. Double DQN separates action selection by the online network from value evaluation by the target network, as in Equation 3.

$$\begin{aligned} a^{*} &= \operatorname{argmax}_{a \in \mathcal{A}(x')} Q_{\theta}(x', a), \\ Y &= r + \gamma (1 - \text{done}) Q_{\bar{\theta}}(x', a^{*}). \end{aligned} \tag{3}$$

Table 1 reports the training and evaluation settings. All networks use a fixed 24,000-transition budget, rather than a checkpoint chosen for favorable test performance. Nine networks are trained: naive/probe, aware/probe and aware/correctness, each with seeds 101, 203 and 307. The probe reward combines a per-step cost of -0.005c(a) and the sampled fraction correct on twelve independent terminal probes. Four probes correspond to each skill and contain no familiarity bonus. The correctness ablation replaces the terminal probe reward with y/12 at each practice step.

Table 1. Training and evaluation settings.

Setting	Value
Catalog / executable tests	12 tasks / 96 cases
History length / attempt cap	12 / 3
Filter histories per seed	300 train; 100 validation
Training seeds	101, 203, 307
Network / parameters	28-64-64-12 ReLU / 6,796
Transitions / networks	24,000 per network / 9 networks
Replay / batch / warm-up	12,000 / 64 / 512 transitions
Update / target copy	Every 4 / 400 transitions
Optimizer / learning rate	Adam / 0.001; Huber loss; norm clip 5
Discount / exploration	0.99; epsilon 1 to 0.05 over 75% of budget
Evaluation	7 policies; 5 worlds; 300 learners/world
Sampling / intervals	1,500 learner identities; 1,500 paired bootstraps
Recorded framework	PyTorch 2.12.0+cpu; one CPU thread

The cost c(a) = 1 + 0.2d is an authored simulation assumption, not measured student solution time. The policy observes sampled response outcomes and reward, not the simulator's hidden mastery. The learner model is fitted separately and held fixed during policy training. Policy and filter errors can therefore affect the resulting behavior, even when the representation has a lower response-prediction loss.

4. Experimental design

4.1. Environments and data partitions

The simulator represents three binary latent skills. Nominal dynamics use learning 0.10, familiarity 0.60 and forgetting 0.005, with difficulty-dependent learning and response emissions. We evaluate five declared environments: nominal, repair-only, slow transfer, increased forgetting and prerequisites. Repair-only sets learning and forgetting to zero while increasing familiarity to 0.80. Slow transfer reduces learning to 0.025; increased forgetting raises forgetting to 0.05. The prerequisite environment reduces the learning rate by a factor of ten when the preceding skill is absent. These are authored stress tests, not empirically calibrated models of student behavior.

For each training seed, the filter uses 300 synthetic training histories and 100 separate validation histories, each containing twelve interactions. Policy training uses a separate seed namespace and nominal dynamics. Evaluation uses 300 new learner seeds in each environment, shared across policies and training fits to enable paired comparisons. Figure 2 makes the sampling unit and the measurement boundary explicit.

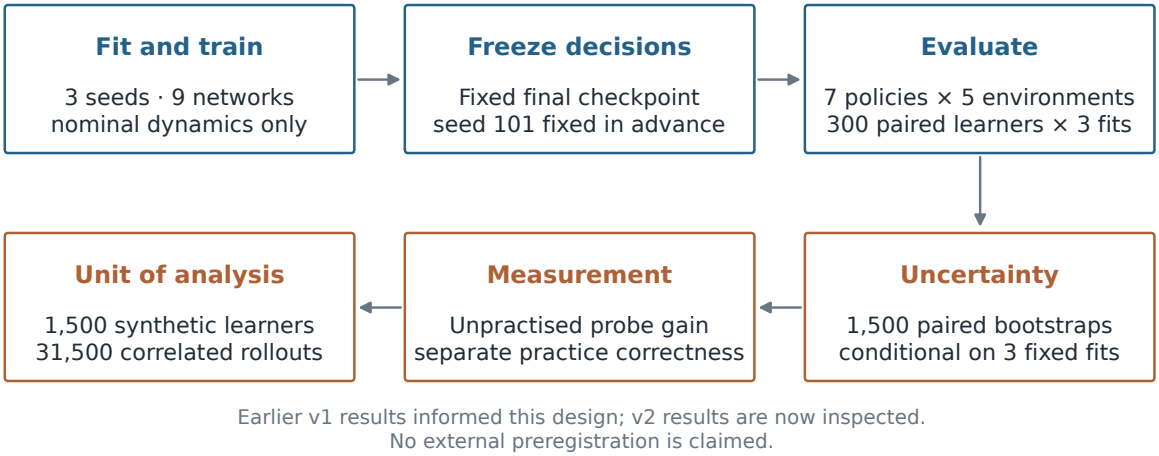


Figure 2. Fixed-design experiment and sampling units. Repeated rollouts share learner seeds and trained fits; they are not independent human participants.

The total of 31,500 rollouts is 7 policies × 5 environments × 3 fits × 300 learners. It comprises 1,500 distinct simulated learner identities, not 31,500 independent students. Source, input hashes, design, commands and outputs are stored in the frozen run. Earlier v1 results had already been inspected and informed v2. The v2 design was recorded before its own evaluation, but this local record is not external preregistration. The reported v2 results are now inspected.

4.2. Comparators and outcomes

The four non-RL comparators select an eligible task randomly, by increasing authored difficulty, by the weakest estimated skill, or by predicted response probability closest to 0.7. The last is a heuristic comparator, not a validated operationalization of the zone of proximal development. The three learned variants provide encoder and reward comparisons under the same fixed interaction budget.

The primary outcome is the change in expected success on an independent transfer probe, defined in Equation 4. Only the held-out evaluator reads latent mastery M to compute this low-noise expectation. The term "transfer" therefore denotes an explicitly defined simulator quantity; it does not denote an observed human transfer test. Practice correctness, final expected probe score, sampled terminal probe score and cost are retained separately.

$$G = \frac{1}{3} \sum_{k=1}^{3} [(0.2 + 0.7M_{T,k}) - (0.2 + 0.7M_{0,k})]. \quad (4)$$

For each simulated learner, we average the outcome over the three fixed fitted models. We then resample learners, preserving policy pairing, for 1,500 percentile bootstrap replicates. The 95% intervals are conditional on those fitted models and do not cover all possible retraining variability. The primary contrast is aware/probe minus naive/probe in each environment; weakest-skill and reward-ablation contrasts are secondary. Multiple comparisons are reported without a multiplicity adjustment and interpreted as exploratory.

5. Results

5.1. Prediction improves, selection does not necessarily improve

Table 2 gives every filter-validation result. Repeat-aware modeling reduces validation negative log-likelihood in all three fits. Figure 3 presents this predictive change alongside policy value, using separate axes and units to avoid equating the two measurements.

Table 2. Validation negative log-likelihood per response.

Seed	Naive	Repeat-aware	Aware - naive
101	0.637583	0.622301	-0.015281
203	0.643206	0.631533	-0.011672
307	0.653636	0.621808	-0.031827

In nominal dynamics, the aware/probe policy achieves 17.91 percentage points of expected transfer-probe gain, compared with 18.85 for naive/probe and 19.16 for weakest skill (Table 3). Its difference from naive/probe is -0.93 points with interval [-1.87, -0.05], and its difference from weakest skill is -1.24 points with interval [-2.27, -0.21]. These contrasts do not support superiority of the repeat-aware policy under the tested budget. They also do not establish that a different training budget or representation would produce the same ranking.

Table 3. Nominal policy outcomes.

Policy	Gain (pp)	95% CI (pp)	Practice (%)
Random	16.96	[14.86, 19.13]	51.69
Difficulty order	16.64	[14.50, 18.90]	54.75
Weakest skill	19.16	[17.07, 21.40]	50.76
Target p = 0.7	14.16	[12.13, 16.24]	59.72
DDQN naive / probe	18.85	[16.83, 20.97]	54.29
DDQN aware / probe	17.91	[15.90, 20.03]	53.61
DDQN aware / correct	17.24	[15.30, 19.37]	56.16

The probability-target heuristic has the highest mean practice correctness in this environment, 59.72%, but lower transfer-probe gain than the other reported policies. This illustrates why a practice-success metric cannot replace the declared policy objective in this simulator. It is not evidence that targeting a particular human success rate is generally harmful.

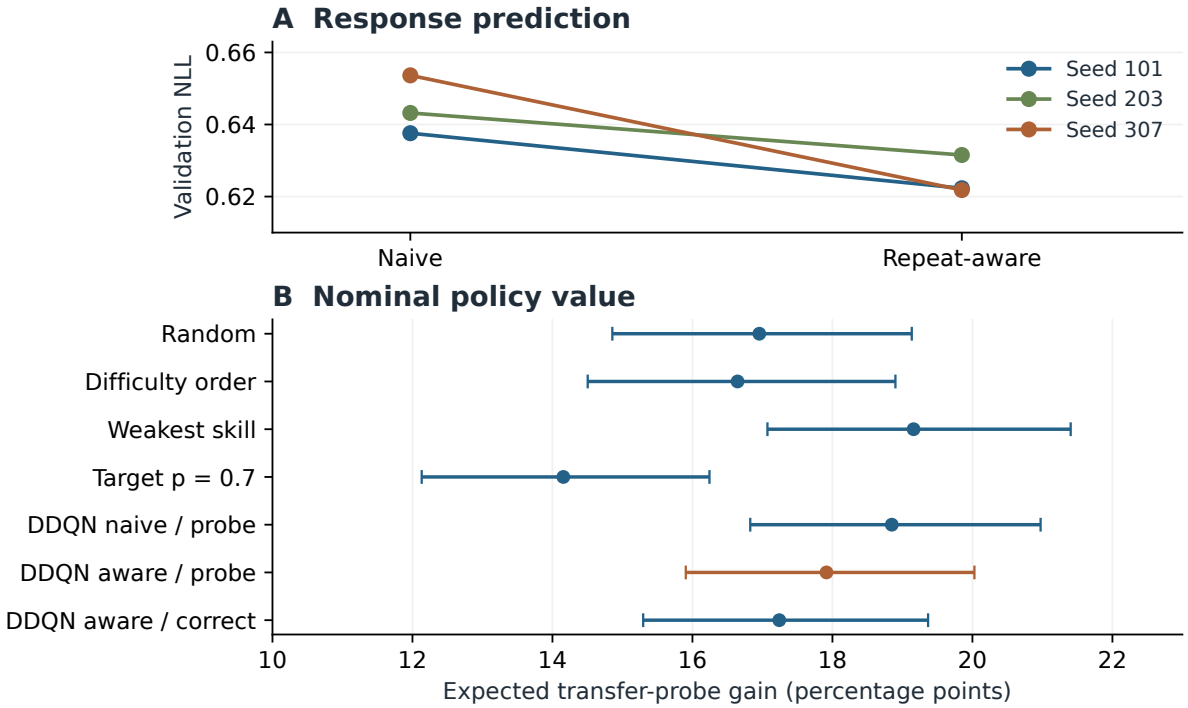


Figure 3. Prediction and selection have different outcomes. A: validation negative log-likelihood, lower is better. B: nominal expected probe gain; bars show 95% learner-bootstrap intervals conditional on three fixed fits.

5.2. Stress tests and reward comparison

Figure 4 reports every environment for the three paired contrasts. The direction of the encoder comparison changes under prerequisites: aware/probe minus naive/probe is 1.19 points with interval [0.16, 2.20]. The comparison with weakest skill in that environment includes zero. Under increased forgetting, aware/probe exceeds its correctness-reward ablation by 2.10 points with interval [1.18, 3.00]. These results are compatible with environment-sensitive policy rankings; they do not isolate a single causal explanation inside the trained networks.

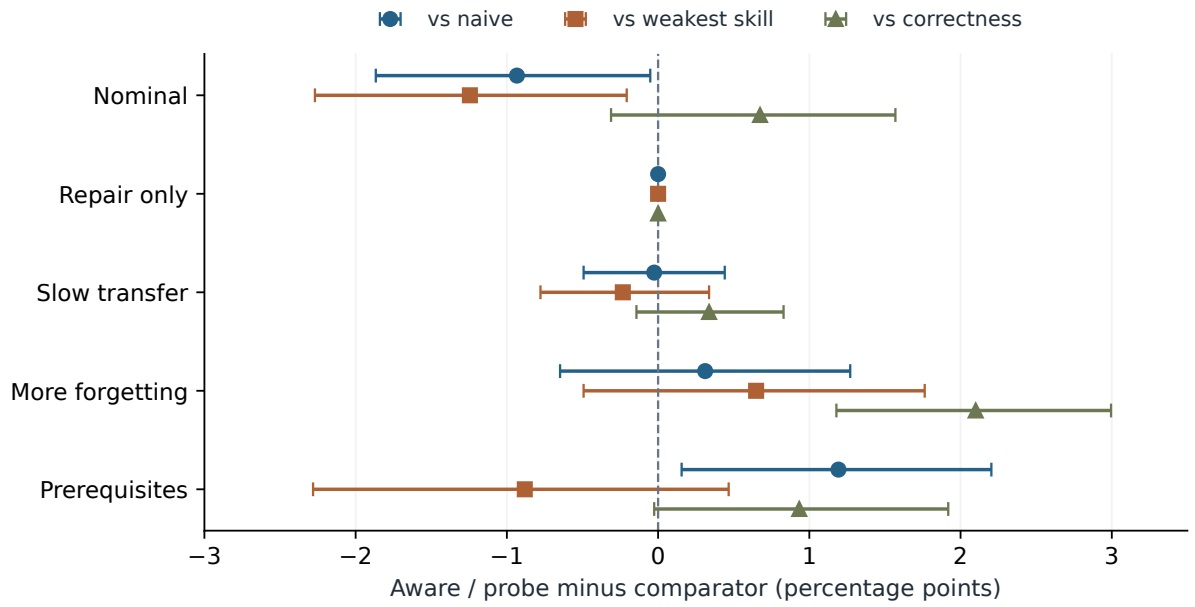


Figure 4. Paired probe-gain contrasts across all five environments. Points compare aware/probe with each named comparator. Bars are conditional 95% intervals; zero means no observed mean difference. No multiplicity adjustment is applied.

In repair-only dynamics, gain is zero for every policy by construction, although practice correctness ranges from 43.75% to 54.86%. The result checks that the implementation can distinguish familiarity-driven success from the chosen transfer measure. The environment deliberately has no new learning; the result must not be described as a discovery that real students cannot learn by retrying. Appendix A retains all 35 policy-environment rows, including zero and unfavorable outcomes, while the companion CSV files provide unrounded means, intervals, fitted parameters and paired contrasts.

6. Application verification

The deployed member is the aware/probe network with seed 101, selected in the protocol before policy evaluation for demonstration. It is not the best-performing policy identified after inspection. Python executes inside a disposable local browser worker; the application supplies concrete test feedback, preserves drafts and exports the decision history. A separate research view compares hypothetical histories without editing personal practice records.

Numerical parity fixtures compare Python and JavaScript belief updates, response predictions and Q values. The maximum observed Q difference is 8.05e-08, below the 0.000002 tolerance; belief and probability differences are zero for the five fixtures. The operational regression includes 40 harness, policy and backend checks, and all 96 reference cases in browser Python. Additional rehearsals cover Chromium, Edge and Chrome, local-only networking, missing-resource recovery and restart of a test-owned server. These checks establish the tested software behavior rather than pedagogical effectiveness or exhaustive correctness for every possible program.

The learned policy has a finite operating scope. Twelve retained observations, excessive attempts on one task, a new observation after a task was solved, no eligible action, or unavailable/incompatible artifacts trigger an explicitly labeled rule fallback. Repeating a solved task manually remains possible. The policy-value tables do not evaluate an unlimited mixture of learned and fallback recommendations in human use.

7. Discussion and limitations

7.1. Interpretation of the negative result

The central finding concerns measurement alignment. Familiarity is useful for predicting repeated-item responses, but a policy trained through that representation must also learn to select actions that advance the chosen objective. Improved likelihood does not by itself certify that this downstream optimization succeeds. A transparent heuristic remains a demanding comparator in the nominal experiment, and retaining that result prevents a prediction gain from being promoted into an unsupported recommendation claim.

The study does not show that repeat-aware models are intrinsically worse, that reinforcement learning is unnecessary, or that the weakest-skill rule is universally optimal. Only three fitted models and one fixed interaction budget were evaluated. No convergence study, broad hyperparameter search or sensitivity analysis over skill mappings was completed. The intervals are conditional and the several comparisons are exploratory.

7.2. Construct and external validity

Latent skills, item difficulties, costs and learning mechanisms are authored assumptions. The fitted model shares part of the simulator's structure, and the five environments cannot represent every source of misspecification. Catalog identity checks validate task mapping in software, not a psychological interpretation of the skill variables. Twelve introductory tasks are not a representative programming curriculum, and passing their finite tests does not demonstrate general problem-solving ability.

The filter does not model source-code semantics, partial test success, hint use, elapsed writing time or forgetting over calendar days. A learner who copies a solution can produce a passing response without the learning implied by a naive interpretation of correctness. Those omitted factors limit use of the current belief values as student-facing diagnostic scores. In particular, the simulated cost does not satisfy an empirical claim of improved human solution time.

No prospective student intervention or validated human off-policy value estimate was obtained. The historical ACcoding forecasting component concerns a fixed logged next task, not the effect of selecting a different one. It cannot validate this authored-bank policy. Modern external recommendation baselines have not been reproduced under a common benchmark, so no claim of algorithmic novelty, SOTA performance or publication-tier readiness follows from the present results.

7.3. Research and design implications

Future evaluation should use public data with genuinely matched tasks and histories, validate skill annotations, and test modern recommendation baselines under a common protocol. A simulator should be fitted and checked against external observations before stronger claims about real learning are made. Any follow-up selected after seeing these results needs a newly declared protocol. These directions do not require an unplanned claim that a student trial has already occurred.

For system design, the practical implication is to expose the provenance and scope of each displayed quantity. Test correctness is executable feedback; predicted success is a model output; belief is an internal state; and transfer-probe gain is a simulator evaluation target. Keeping these meanings distinct makes a demonstration inspectable without converting its visual plausibility into evidence of learning impact.

8. Conclusion

The prototype connects executable programming tasks to learner-state updates and learned exercise selection. Its controlled experiment finds better repeat-aware response likelihood in every fitted seed, but no corresponding nominal advantage over the naive policy or weakest-skill comparator. Independent probes, stress tests, artifact checks and software rehearsals make the claim auditable. The demonstrated result is a functioning implementation and a bounded simulator finding; human learning effectiveness remains unestablished.

Data and code availability

The local artifact contains the catalog, nine trained networks, protocol and source snapshots, rollout arrays, aggregate summaries, decision traces and reporting scripts. It has not been assigned a public archive DOI, and this draft does not imply that the artifact is already publicly deposited. Raw historical learner records are excluded from the new application bundle. The reporting pipeline verifies the frozen run, checks aggregate means against rollout arrays and regenerates figures and editable tables without refitting a model. The recorded experiment took 482.0 seconds on the available CPU environment; this is not a hardware-comparable efficiency benchmark.

Ethics and disclosure status

The policy experiment uses synthetic learners, and software verification uses disposable test histories. No new student intervention was conducted. Historical public-log work is a separate component and does not provide evidence of informed consent, institutional approval or an exemption for this manuscript. Any applicable institutional determination, funding statement, competing-interest declaration and author-contribution statement require confirmation by the human authors before submission; none is invented here.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex assisted with programming, experiment automation, literature retrieval, manuscript drafting and reproducible plotting code. Numerical results are read from recorded artifacts. Figures are deterministic vector plots and diagrams produced by code; they are not image-model renderings. Final human review of all claims, references, declarations and the journal's current AI policy remains required before submission. This working draft does not represent that such review has already been completed.

References

Cheng, X., Zhuang, G., Xie, Y., Wang, J., Liu, Y., Guan, Q., Fang, L., & Pan, S. (2026). *UniER: A unified benchmark for item-level and path-level exercise recommendation* [Preprint]. arXiv. https://arxiv.org/abs/2605.16750

Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction, 4*, 253-278. https://doi.org/10.1007/BF01099821

Girard, S., Minn, S., Bouzeghoub, A., & Vie, J.-J. (2026). *Counterfactual learning of new adaptive instructional policies using logged data* [Preprint]. arXiv. https://arxiv.org/abs/2606.23015

Minn, S. (2022). AI-assisted knowledge assessment techniques for adaptive learning environments. *Computers and Education: Artificial Intelligence, 3*, Article 100050. https://doi.org/10.1016/j.caeai.2022.100050

Ozyurt, Y., Almaci, T., Feuerriegel, S., & Sachan, M. (2025). Personalized exercise recommendation with semantically-grounded knowledge tracing. *Advances in Neural Information Processing Systems, 38*. https://proceedings.neurips.cc/paper_files/paper/2025/hash/13707aad517ddd6c09ea02e0f55e1e7a-Abstract-Conference.html

Piech, C., Spencer, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems, 28*. https://arxiv.org/abs/1506.05908

Shi, Y., Chi, M., Barnes, T., & Price, T. W. (2022). *Code-DKT: A code-based knowledge tracing model for programming tasks* [Preprint]. arXiv. https://arxiv.org/abs/2206.03545

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 30*(1), 2094-2100. https://doi.org/10.1609/aaai.v30i1.10295

Appendix A. Complete policy outcomes

The table preserves the full design instead of selecting only favorable environments. Gain and confidence intervals are in percentage points; practice correctness is a percentage. Intervals are conditional paired-learner bootstrap intervals, with three fixed fitted models and 300 learners per environment. The companion tables include the unrounded quantities and all paired contrasts.

Table A.1. Complete policy-environment outcomes.

Environment	Policy	Gain (pp)	95% CI (pp)	Practice (%)
Nominal	Random	16.96	[14.86, 19.13]	51.69
	Difficulty order	16.64	[14.50, 18.90]	54.75
	Weakest skill	19.16	[17.07, 21.40]	50.76
	Target p = 0.7	14.16	[12.13, 16.24]	59.72
	DDQN naive / probe	18.85	[16.83, 20.97]	54.29
	DDQN aware / probe	17.91	[15.90, 20.03]	53.61
	DDQN aware / correct	17.24	[15.30, 19.37]	56.16
Repair only	Random	0.00	[0.00, 0.00]	43.75
	Difficulty order	0.00	[0.00, 0.00]	47.69
	Weakest skill	0.00	[0.00, 0.00]	46.10
	Target p = 0.7	0.00	[0.00, 0.00]	54.86
	DDQN naive / probe	0.00	[0.00, 0.00]	49.16
	DDQN aware / probe	0.00	[0.00, 0.00]	48.68
	DDQN aware / correct	0.00	[0.00, 0.00]	50.19
Slow transfer	Random	4.67	[3.34, 5.99]	43.50
	Difficulty order	4.59	[3.34, 5.99]	46.36
	Weakest skill	5.39	[4.03, 6.94]	42.97
	Target p = 0.7	3.89	[2.72, 5.21]	53.33
	DDQN naive / probe	5.19	[3.97, 6.59]	46.43
	DDQN aware / probe	5.16	[3.89, 6.51]	46.26
	DDQN aware / correct	4.82	[3.60, 6.16]	49.17
More forgetting	Random	8.56	[6.07, 11.12]	48.64
	Difficulty order	8.24	[5.68, 10.97]	50.33
	Weakest skill	8.61	[6.14, 11.16]	48.38
	Target p = 0.7	6.33	[3.85, 8.79]	56.71
	DDQN naive / probe	8.94	[6.70, 11.30]	50.56
	DDQN aware / probe	9.26	[6.96, 11.68]	50.21
	DDQN aware / correct	7.16	[4.87, 9.44]	51.92
Prerequisites	Random	10.66	[8.79, 12.60]	46.58
	Difficulty order	11.04	[9.41, 12.76]	50.25
	Weakest skill	12.21	[10.41, 14.21]	47.44
	Target p = 0.7	8.89	[7.30, 10.62]	55.29
	DDQN naive / probe	10.14	[8.66, 11.69]	48.70
	DDQN aware / probe	11.33	[9.83, 12.81]	49.66
	DDQN aware / correct	10.40	[8.93, 12.00]	51.30